

# Electric Vehicle Routing Problem with Multi-Service Delivery Model

Submitted by: Vidhi Sonkar (25999), Anant Rajput (27252)

## 1 Introduction

The rapid expansion of e-commerce has intensified the importance of efficient last-mile delivery. This final segment of the logistics chain plays a critical role in customer experience and represents a major operational cost component, accounting for nearly half of total delivery expenses. In response to these growing economic pressures, logistics providers are increasingly exploring sustainable and cost-effective transportation technologies. Electric vehicles (EVs) have emerged as a compelling alternative due to their reduced energy cost, lower maintenance requirements, and alignment with environmental and emission regulations. Their use in last-mile distribution is expected to increase substantially, with projections indicating close to 8 million EVs in operation by 2030.

Despite these advantages, widespread adoption of EVs remains limited by high initial investment costs and insufficient availability of rapid charging infrastructure. These constraints impact route planning flexibility and reduce vehicle utilization efficiency.

In addition to their mobility benefits, EVs provide a new operational opportunity through their onboard battery storage. Beyond powering transportation, stored energy can be discharged to the electrical grid during peak demand periods via vehicle-to-grid (V2G) mechanisms. When market prices differ between charging and discharging, this creates an additional revenue channel for logistics operators. Such a dual-service operational model—encompassing goods delivery and energy delivery—has the potential to reduce overall last-mile logistics costs.

This work investigates whether EV fleets can simultaneously satisfy traditional delivery requirements while participating in V2G services without compromising primary delivery objectives. We formulate the problem as an electric vehicle routing problem (EVRP) with time windows and energy discharge, where vehicles must not only visit customers but also determine advantageous times and locations for energy discharge. The problem is challenging due to limited battery resources, strict delivery time constraints, and the need to align both delivery and V2G operations spatially and temporally. Classical optimization techniques, such as mixed-integer linear programming (MILP) and metaheuristics including particle swarm optimization, genetic algorithms, tabu search, simulated annealing, and ant colony optimization, face scalability limitations when applied to large instances involving hundreds of customers and stations.

## 2 Problem Description and Operational Framework

In this section, we describe the constrained electric vehicle routing problem with time windows and energy discharge (CEVRPTW-D), along with the operational setting and modeling assumptions for routing EVs that support vehicle-to-grid (V2G) energy delivery.

### 2.1 Operational Scenario

We consider a last-mile delivery environment in which a logistics operator must plan routes for a fleet of electric vehicles (EVs) ahead of their departure. All required information, including cus-

tomers orders, delivery time windows, and the duration of peak energy demand on the electrical grid, is assumed to be known prior to scheduling. Each EV leaves the depot with an initial state-of-charge (SoC) denoted by  $q_u$ , completes its assigned deliveries, and must return to the depot with no less than 20% remaining battery. During operation, vehicles may optionally visit discharging stations to sell energy back to the grid during the specified peak interval, provided that delivery performance is not adversely affected.

### 2.2 Modeling Assumptions

The ESEVRPTW formulation is evaluated using the Solomon dataset under a deterministic routing setting. Customer and discharging station coordinates are represented in a Euclidean plane, where travel distances and times are derived directly from geometric separation, implying symmetric travel and no variability due to road-network constraints. Customer service windows, service durations, and demand quantities are treated as fixed parameters, and vehicles may wait at a location if they arrive early but may not violate the assigned time windows. All EVs are assumed to be identical with respect to cargo capacity, battery capacity, and energy consumption characteristics. Energy consumption is modeled as linearly proportional to travel distance, and the discharge of stored energy to the grid occurs only within predefined peak pricing intervals. Effects such as battery degradation, charging inefficiency, and dynamic electricity pricing are not considered.

All routes originate and terminate at a single depot, and each customer is visited exactly once. Decision making is assumed to occur under full information, without stochastic influences. Although our model assumes deterministic travel and battery dynamics, prior work has examined stochastic extensions that incorporate uncertainty in energy usage. These assumptions simplify complexities such as unpredictable traffic patterns, heterogeneous vehicle behavior, and fluctuating energy conditions, enabling a tractable foundation for reinforcement-learning-based policy development.

### 2.3 System Model

The operational network is represented as a complete undirected graph  $G = (V, E)$ , where the set of nodes is defined as:

$$V = \{v_0\} \cup K \cup P$$

Here,  $v_0$  represents the depot;  $K = \{k_1, \dots, k_m\}$  is the set of customer nodes; and  $P = \{p_1, \dots, p_n\}$  represents the set of discharging stations where V2G operations can occur. The fleet stationed at the depot consists of homogeneous EVs, denoted by  $X = \{x_1, \dots, x_u\}$ , each vehicle having a load capacity  $C$ , a maximum battery capacity  $Q$ , and initial SoC  $q_u$ .

Each customer  $k_i \in K$  is associated with a delivery demand  $c_i$ , a service duration  $s_i$ , and a time window  $[e_i, l_i]$ . Every edge  $(i, j) \in E$  is associated with a travel distance  $d_{ij}$ , travel time  $t_{ij}$ , and travel energy consumption  $b_{ij}$ .

**Table 1**

Notation and Variable Definitions

| Symbol                      | Description                     |
|-----------------------------|---------------------------------|
| <b>Time</b>                 |                                 |
| $t$                         | Time index for decision-making  |
| $[1, T]$                    | Planning horizon                |
| <b>Sets</b>                 |                                 |
| $V = \{v_0\} \cup K \cup P$ | Node set                        |
| $K = \{k_1, \dots, k_m\}$   | Customer nodes                  |
| $P = \{p_1, \dots, p_n\}$   | Discharging station nodes       |
| $X = \{x_1, \dots, x_u\}$   | Electric vehicles               |
| <b>Customer</b>             |                                 |
| $e_i$                       | Earliest service start time     |
| $l_i$                       | Latest service start time       |
| $s_i$                       | Service duration                |
| $c_i$                       | Delivery demand                 |
| <b>Travel</b>               |                                 |
| $d_{ij}$                    | Distance between nodes $(i, j)$ |
| $t_{ij}$                    | Travel time $(i, j)$            |
| $H$                         | Consumption rate (kWh/km)       |
| $b_{ij}$                    | Energy used $H \cdot d_{ij}$    |
| <b>Vehicle</b>              |                                 |
| $C$                         | Vehicle load capacity           |
| $Q$                         | Battery capacity                |
| <b>Grid / V2G</b>           |                                 |
| $g_i^1$                     | Peak start time at station $i$  |
| $g_i^2$                     | Peak end time at station $i$    |
| $\gamma_i$                  | Discharging service time        |
| $R$                         | Discharge rate                  |
| <b>Decision Variables</b>   |                                 |
| $\alpha_{ij}^u$             | 1 if EV $u$ uses edge $(i, j)$  |
| $\tau_i^u$                  | Arrival time of EV $u$          |
| $\theta_i^u$                | Battery level on arrival        |
| $\lambda_i^u$               | Remaining cargo                 |

Battery energy is consumed in two forms: (a)  $H$  (kWh/km) during travel between nodes, and (b)  $R$  (kWh) discharged when selling energy to the grid during the peak period  $[G_{i1}, G_{i2}]$  at discharging station  $i$ . We assume a single peak-demand interval per planning horizon, and therefore each EV discharges only once per day. A summary of notation is provided in Table 1.

### 3 Optimization Problem and MILP Formulation

The CEVRPTW-D is modeled as a Mixed-Integer Linear Programming (MILP) problem designed to jointly optimize routing logistics, battery management, and vehicle-to-grid (V2G) scheduling 56. The objective function is constructed to minimize total operational costs while maximizing revenue generated from energy discharge.

The economic parameters are calibrated based on the operational context of Bangalore, Karnataka. The average EV charging cost is INR 8/kWh 58. Using a Mahindra E-Supro as the reference vehicle (approximately 8 km/kWh efficiency), the travel cost coefficient is estimated at  $Y_1 = 1$  INR/km 59. The fixed vehicle leasing cost is set to  $Y_2 = 700$  INR/day 60. Revenue from discharging energy to the grid is calculated at a rate of  $Y_3 = 4.3$  INR/kWh 61.

The mathematical formulation is defined as follows:

$$\mathcal{M} = \min \left( Y_1 \sum_{u \in X} \sum_{i \in V} d_{ij} \alpha_{ij}^u + Y_2 \sum_{u \in X} \sum_{i \in V} \alpha_{0i}^u - Y_3 \sum_{u \in X} \sum_{i \in P} \gamma_i^u \sum_{j \in V} \alpha_{ij}^u \right)$$

**Subject to:**

$$\sum_{u \in X} \sum_{j \in V, j \neq i} \alpha_{ij}^u = 1, \quad \forall i \in K. \quad (2)$$

$$\sum_{j \in V, j \neq i} \alpha_{ij}^u = \sum_{k \in V, k \neq i} \alpha_{ji}^u, \quad \forall j \in V, \forall u \in X. \quad (3)$$

$$t_i^u + (t_{ij} + s_i) \alpha_{ij}^u - l_0(1 - \alpha_{ij}^u) \leq t_j^u, \quad \forall i \in K, j \in V, u \in X. \quad (4)$$

$$t_i^u + (t_{ij} + \gamma_i) \alpha_{ij}^u - l_0(1 - \alpha_{ij}^u) \leq t_j^u, \quad \forall i \in P, j \in V, u \in X. \quad (5)$$

$$e_i \sum_{j \in V} \alpha_{ij}^u \leq t_i^u \leq l_i \sum_{j \in V} \alpha_{ij}^u, \quad \forall i \in P, u \in X. \quad (7)$$

$$g_i^1 \sum_{j \in V} \alpha_{ij}^u \leq t_i^u \leq g_i^2 \sum_{j \in V} \alpha_{ij}^u, \quad \forall i \in P, u \in X. \quad (8)$$

$$0 \leq \theta_i^u \leq \theta_i^u - (Hd_{ij} + Q) \alpha_{ij}^u + Q(1 - \alpha_{ij}^u), \quad \forall i \in K, j \in V, u \in X, i \neq j. \quad (9)$$

$$0 \leq \theta_i^u \leq \theta_i^u - (Hd_{ij} + R\gamma_i^u) \alpha_{ij}^u + Q(1 - \alpha_{ij}^u), \quad \forall i \in P, j \in V, u \in X, i \neq j. \quad (10)$$

$$\lambda_i^u - c_i \alpha_{ij}^u - C(1 - \alpha_{ij}^u) \leq \lambda_j^u - c_i \alpha_{ij}^u + C(1 - \alpha_{ij}^u), \quad \forall i, j \in V, u \in X. \quad (12)$$

$$\theta_0^u = Q, \quad \lambda_0^u = C, \quad \theta_0^u \geq 0.2Q, \quad \forall u \in X. \quad (13-15)$$

#### 3.1 Constraint Analysis

The constraint set ensures physical and temporal feasibility. Equation (2) 104 mandates that every customer is visited exactly once. Equation (3) 105 enforces flow conservation by balancing vehicle arrivals and departures at each node. Temporal feasibility is enforced through Equation (4) for customer service and Equation (5) 107 for discharging stations, accounting for service durations and discharge times.

Equations (7) and (8) 108; 109 enforce time windows for customer deliveries and peak-demand discharge intervals, respectively. Battery state-of-charge dynamics are modeled using Equation (9) for consumption and Equation (10) 110 for reduction due to discharge. Finally, capacity constraints and initial conditions are addressed in Equation (12-14), while Equation (15) 112; 113; 114 ensures a minimum 20% battery reserve upon returning to the depot.

## 4 MILP Solver Integration and Scalability Limitations

To evaluate the formulation, the MILP was implemented and solved using the Gurobi optimizer. The system parses Solomon RC benchmark datasets, constructs a Euclidean spatial graph, and identifies V2G discharge stations **116**; **117**. The model generates a complete operational schedule containing binary routing decisions  $\alpha_{ij}^u$ , arrival timestamps, battery levels, and discharge durations **119**.

However, due to the NP-hard nature of the CEVRPTW-D and the large number of constraints, the exact MILP approach exhibits significant scalability limitations. In our experiments, the solver reliably converged to optimal solutions only for small-scale instances containing up to 10 nodes (including customers and stations). For problem sizes exceeding this limit, computation time increased exponentially and often failed to reach optimality within acceptable runtime.

Consequently, although the MILP formulation provides an exact benchmark and theoretical validation of feasibility, practical computational limits necessitated the development of heuristic (PSO) and learning-based (Rainbow DQN) approaches to solve larger and more realistic logistics networks **26**; **269**.

## 5 PSO-based Metaheuristic for the CEVRPTW-D

In addition to the exact MILP-based formulation, we develop a particle swarm optimization (PSO) metaheuristic to obtain high-quality solutions for the CEVRPTW-D within reasonable computation times. The PSO operates directly on a route-based solution representation and is tailored to jointly handle routing, capacity constraints, time windows, and vehicle-to-grid (V2G) discharge decisions.

### 5.1 Solution Encoding and Fitness Evaluation

A candidate solution consists of a set of vehicle routes, where each route is represented as an ordered sequence of node indices excluding the depot, together with a mapping from discharging station nodes to discharge durations. For each particle, a complete solution aggregates the routes of all vehicles in the fleet. The depot is implicitly added at the beginning and end of every route.

The fitness of a solution is evaluated by simulating all vehicle routes sequentially. For each route, the algorithm computes the total travel distance, cumulative travel time, battery consumption, and cargo evolution along the sequence of visited nodes. Time window constraints at customer nodes are enforced by allowing waiting when arriving early and penalizing late arrivals. Load feasibility is checked against the vehicle capacity, and violations of capacity, battery limits, or time windows incur penalty terms.

At discharging station nodes, the PSO solution specifies a discharge duration, which is applied only within the grid's peak demand interval. The simulation adjusts the vehicle timeline to respect the discharge window and reduces the battery level according to the discharge duration. The total amount of discharge time accumulated over all stations and vehicles contributes to the revenue term in the objective.

After all routes are simulated, the algorithm verifies that every customer is visited exactly once. Missing customers incur a large penalty. The overall fitness of a solution is computed as

$$f = Y_1 \cdot \text{Total Distance} + Y_2 \cdot \text{Vehicles Used} - Y_3 \cdot \text{Total Discharge} + \text{Penalties.}$$

where  $Y_1$ ,  $Y_2$ , and  $Y_3$  are the same weighting coefficients used in the MILP objective. Lower fitness values correspond to better solutions.

### 5.2 Particle Representation and Initialization

Each particle in the swarm maintains:

- a current solution (set of routes and station discharge times),
- a personal best solution and its associated fitness, and
- a velocity, represented as a list of route-editing operations.

To initialize the swarm, we generate heuristic solutions using a nearest-neighbor construction.

Customers are assigned to vehicles iteratively by selecting the nearest feasible customer with respect to remaining capacity, battery level, and delivery time windows. After building a route, an optional discharging station may be appended if time and battery conditions permit beneficial V2G discharge within the peak interval. Any remaining unassigned customers are placed in singleton routes to ensure full coverage.

### 5.3 Velocity, Neighborhood Moves, and Position Update

The PSO dynamics are implemented through route modification operators that act as the “velocity” of a particle. We use several neighborhood moves:

- *swap-within-route*: exchanges two customers within the same route,
- *swap-between-routes*: exchanges customers belonging to different routes,
- *move*: relocates a customer from one route to another, and
- *2-opt*: performs a 2-opt reversal on a segment of a route to reduce travel distance.

The velocity of a particle is a sequence of such operations. At each iteration, a particle's velocity is updated using three components:

1. a cognitive component that generates operations to move the current solution towards the particle's personal best solution,
2. a social component that generates operations to move toward the global best solution found by the swarm, and
3. an inertia component that introduces random perturbations to maintain exploration.

The generated operations are then applied to the current solution to obtain a new solution (position). Empty routes are removed after applying moves, and the new solution is re-evaluated using the fitness function described above. If the new fitness improves the particle's personal best, the personal best is updated. The global best across all particles is updated similarly.

### 5.4 Algorithm Settings and Termination

The PSO solver is parameterized by the number of particles, maximum iterations, and the inertia ( $w$ ), cognitive ( $c_1$ ), and social ( $c_2$ ) coefficients. During the search, the inertia weight is gradually decreased to shift the algorithm from exploration to exploitation. An early-stopping criterion is employed: if the global best fitness does not improve for a fixed number of iterations, the algorithm terminates.

---

**Algorithm 1** PSO for CEVRPTW-D

---

**Require:** Problem instance  $(V, K, P)$ , number of vehicles  $U$ , battery capacity  $Q$ , number of particles  $N_p$ , maximum iterations  $N_{\text{iter}}$ , weights  $Y_1, Y_2, Y_3$ , PSO parameters  $w, c_1, c_2$

**Ensure:** Best solution  $S^{\text{gbest}}$  and fitness  $f^{\text{gbest}}$

- 1: Initialize global best fitness  $f^{\text{gbest}} \leftarrow +\infty$
- 2: Initialize empty global best solution  $S^{\text{gbest}}$
- 3: **for**  $p = 1$  to  $N_p$  **do**
- 4:   Generate initial solution  $S_p \leftarrow \text{RANDOMINITSOLUTION}()$
- 5:   Compute fitness  $f_p \leftarrow \text{EVALUATESOLUTION}(S_p)$
- 6:   Set particle best  $S_p^{\text{pbest}} \leftarrow S_p, f_p^{\text{pbest}} \leftarrow f_p$
- 7:   Initialize velocity  $V_p \leftarrow \emptyset$  {list of route-edit operations}
- 8:   **if**  $f_p < f^{\text{gbest}}$  **then**
- 9:      $S^{\text{gbest}} \leftarrow S_p, f^{\text{gbest}} \leftarrow f_p$
- 10:   **end if**
- 11: **end for**
- 12:  $\text{no\_improve} \leftarrow 0$
- 13: **for**  $\text{iter} = 1$  to  $N_{\text{iter}}$  **do**
- 14:   **for**  $p = 1$  to  $N_p$  **do**
- 15:      $S_p \leftarrow$  current solution of particle  $p$
- 16:     // **Velocity update**
- 17:      $V_p \leftarrow \emptyset$
- 18:     **if**  $\text{rand}() < c_1$  **then**
- 19:        $V_p \leftarrow V_p \cup \text{SWAPSTOWARD}(S_p, S_p^{\text{pbest}})$
- 20:     **end if**
- 21:     **if**  $\text{rand}() < c_2$  **then**
- 22:        $V_p \leftarrow V_p \cup \text{SWAPSTOWARD}(S_p, S^{\text{gbest}})$
- 23:     **end if**
- 24:     **if**  $\text{rand}() < w$  **then**
- 25:        $V_p \leftarrow V_p \cup \text{RANDOMMOVE}(S_p)$
- 26:     **end if**
- 27:     // **Position update**
- 28:      $\tilde{S}_p \leftarrow \text{APPLYVELOCITY}(S_p, V_p)$
- 29:     Remove empty routes from  $\tilde{S}_p$
- 30:      $\tilde{f}_p \leftarrow \text{EVALUATESOLUTION}(\tilde{S}_p)$
- 31:      $S_p \leftarrow \tilde{S}_p, f_p \leftarrow \tilde{f}_p$
- 32:     // **Update personal best**
- 33:     **if**  $f_p < f_p^{\text{pbest}}$  **then**
- 34:        $S_p^{\text{pbest}} \leftarrow S_p, f_p^{\text{pbest}} \leftarrow f_p$
- 35:     **end if**
- 36:     // **Update global best**
- 37:     **if**  $f_p < f^{\text{gbest}}$  **then**
- 38:        $S^{\text{gbest}} \leftarrow S_p, f^{\text{gbest}} \leftarrow f_p$
- 39:      $\text{no\_improve} \leftarrow 0$
- 40:   **end if**
- 41: **end for**
- 42:   Record  $f^{\text{gbest}}$  in convergence history
- 43:    $\text{no\_improve} \leftarrow \text{no\_improve} + 1$
- 44:   **if**  $\text{no\_improve} > 50$  **then**
- 45:     **break** {Early stopping}
- 46:   **end if**
- 47:    $w \leftarrow \max(0.4, 0.99 \cdot w)$  {Adaptive inertia}
- 48: **end for**
- 49: **return**  $S^{\text{gbest}}, f^{\text{gbest}}$

---

At termination, the global best solution provides a set of vehicle routes and associated discharge schedules that approximate the optimal trade-off between distance traveled, number of vehicles used, and V2G revenue. This PSO-based metaheuristic serves as a scalable baseline for the CEVRPTW-D and complements the exact MILP approach in scenarios where problem size or real-time requirements preclude solving the full optimization model to optimality.

## 6 Rainbow DQN

To scale beyond exact solvers and metaheuristics for large CEVRPTW-D instances, we adopt a deep reinforcement learning policy based on the Rainbow Deep Q-Network (Rainbow DQN). Rainbow combines six complementary improvements over the original DQN – Double DQN, Dueling Networks, Prioritized Experience Replay, multi-step bootstrap returns, NoisyNet exploration, and distributional value estimation (C51) – producing a stable, sample-efficient learner suitable for long-horizon routing with V2G decisions. This agent-based RL approach follows recent work applying RL to EV routing and V2G scheduling. [Narayanan et al., 2024a](#); [Narayanan et al., 2024b](#); [Bruni et al., 2023](#)

### 6.1 Environment and state representation

We model the CEVRPTW-D as an episodic Markov decision process implemented as a Gymnasium environment. The agent constructs routes sequentially: at each decision step it controls the currently active vehicle and chooses the next feasible node (customer, V2G station, or depot). To remain computationally efficient, observations are compact per-candidate feature vectors (one vector per feasible node) rather than a dense global tensor.

**Action masking.** The environment computes a feasibility mask  $M_t$  at every step to enforce hard constraints. A target node  $j$  is feasible only if:

- **Battery feasibility:** remaining SoC permits travel to  $j$ , performing service/discharge (if any), and reaching a charging station or depot while satisfying minimum return SoC;
- **Time-window feasibility:** arrival and service finish fit within  $[a_j, b_j]$ ;
- **Capacity feasibility:** remaining cargo capacity suffices for customer demand;
- **Discharge feasibility:** for V2G nodes, arrival aligns with the station’s discharge time-window and grid pricing peaks.

**Per-node state vector.** For each feasible node  $j$  we provide a normalized vector

$$s_t^j = [E_{\text{req}}, E_{\text{gain}}, \mathbb{I}_{\text{depot}}, \mathbb{I}_{\text{type}}, \Delta t_{\text{wait}}],$$

where  $E_{\text{req}}$  is the energy required to reach  $j$ ,  $E_{\text{gain}}$  is potential V2G discharge energy (zero for customers),  $\mathbb{I}_{\text{depot}}$  and  $\mathbb{I}_{\text{type}}$  are binary indicators, and  $\Delta t_{\text{wait}}$  is waiting time until the time-window opens. This compact encoding captures vehicle-centric and node-centric information while keeping the action-value network lightweight.

---

**Algorithm 2** Rainbow DQN training for CEVRPTW-D

---

**Input:** EVRP instance  $I$ , max episodes  $K$ , target update freq.  $C$ , batch size  $B$ , learning rate  $\eta$

**Input:** Rainbow params: atoms  $N_{\text{atoms}}$ , multi-step  $n$ , discount  $\gamma$

**Output:** Trained network  $Q_\theta$ , training logs

Initialize online network  $Q_\theta$  and target  $Q_{\theta^-} \leftarrow Q_\theta$  Initialize prioritized replay buffer  $\mathcal{D}$  and  $n$ -step buffer  $\mathcal{N}$  **for** episode  $k \leftarrow 1$  **to**  $K$  **do**

    Sample / build EVRP instance;  $s \leftarrow \mathcal{E}.\text{reset}()$ ; clear  $\mathcal{N}$ ;

$R_{\text{ep}} \leftarrow 0$  **while** feasible actions exist **do**

        Compute action mask  $\mathcal{M}_t \leftarrow \mathcal{E}.\text{get\_feasible\_pairs}()$  Reset NoisyNet parameters in  $Q_\theta$  Select action  $a_t = \arg \max_{a \in \mathcal{M}_t} \mathbb{E}[Z_\theta(s_t, a)]$  Execute  $a_t$ , observe  $(r_t, s_{t+1}, \text{done})$ ; push  $(s_t, a_t, r_t, s_{t+1})$  to  $\mathcal{N}$ ;  $R_{\text{ep}} += r_t$

**if**  $|\mathcal{N}| = n$  **then**

        Compute  $n$ -step return  $R^{(n)} = \sum_{i=0}^{n-1} \gamma^i r_{t+i}$  and store  $(s_t, a_t, R^{(n)}, s_{t+n}, \text{done})$  in  $\mathcal{D}$

**end**

**if**  $|\mathcal{D}| > B$  **then**

        Sample mini-batch from  $\mathcal{D}$  using PER weights  
        Compute distributional target, project onto atom support and minimize KL loss; update priorities by TD-error Gradient step on  $\theta$  with learning rate  $\eta$  **if**  $\text{training step} \bmod C = 0$  **then**

$Q_{\theta^-} \leftarrow Q_\theta$

**end**

**end**

**if** done **then**

        flush remaining  $\mathcal{N}$  to  $\mathcal{D}$ ; **break**

**end**

$s_t \leftarrow s_{t+1}$

**end**

Log episode metrics (cost, distance, vehicles used, V2G revenue)

**end**

**return**  $Q_\theta$ , training logs

---

## 6.2 Reward shaping

To align local decisions with the global objective (minimize cost, maximize V2G revenue) we use a shaped per-step reward:

$$R_t = \rho_{\text{serve}} \mathbb{I}_{\text{cust}} + \rho_{\text{v2g}} E_{\text{dis}} - \rho_{\text{dist}} d_{ij} - \rho_{\text{wait}} \max(0, a_j - T_{\text{arr}}) - \rho_{\text{veh}} \mathbb{I}_{\text{new}}.$$

Here  $\rho_{\text{serve}}, \rho_{\text{v2g}} > 0$  encourage serving customers and discharging energy;  $\rho_{\text{dist}}, \rho_{\text{wait}} > 0$  penalize distance and idle waiting;  $\rho_{\text{veh}}$  penalizes deploying a new vehicle to encourage compact fleet use. A post-episode evaluation metric computes the formal cost used for comparison to MILP/heuristics:

$$\text{Cost} = Y_1 \cdot (\text{total distance}) + Y_2 \cdot (\text{vehicles used}) - Y_4 \cdot (\text{total discharged energy}).$$

## 6.3 Network architecture and training details

We adopt a dueling distributional network that outputs a discrete return distribution over  $N_{\text{atoms}}$  support atoms in  $[V_{\min}, V_{\max}]$ . The network first encodes per-node feature vectors via a shared MLP, then splits into value and advantage streams (dueling head). NoisyNet layers replace standard  $\epsilon$ -greedy exploration so the agent learns parametric, state-dependent exploration. Training minimizes the KL divergence between the predicted categorical distribution and the projected distributional target using prioritized replay sampling and  $n$ -step bootstrap targets to accelerate learning along long routes. Hyperparameters (atoms,  $n$ , PER parameters, optimizer settings) are tuned on small Solomon-like instances, then transferred to larger training instances during curriculum learning. This setup follows the growing literature on RL for EV routing and V2G-enabled operations. [Narayanan et al., 2024a](#); [Narayanan et al., 2024b](#); [Bruni et al., 2023](#)

## 6.4 Remarks

The Rainbow DQN policy offers a reusable routing heuristic: once trained, the agent can quickly construct feasible routes and exploit V2G opportunities in real time, providing a practical complement to MILP and population-based heuristics for large-scale, stochastic EVRP variants.

## 7 Interpretations

This section presents a detailed comparative analysis of the Particle Swarm Optimization (PSO) approach and the Rainbow Deep Q-Network (Rainbow DQN) for the Electric Vehicle Routing Problem with Time Windows and V2G integration. The comparison is based on several performance indicators, including operational cost, fleet size, travel distance, energy management, and computational time. The results highlight clear distinctions between the global metaheuristic (PSO) and the constructive reinforcement learning agent (Rainbow DQN).

### 7.1 Cost Efficiency

The results indicate that PSO achieves significantly lower operational costs compared to the Rainbow DQN. On average, the solutions generated by the DQN are approximately 99.8% more expensive than the PSO solutions. The primary reason for this difference lies in the formulation of the cost function, which includes a heavy penalty term associated with the number of vehicles used. Since PSO optimizes the entire schedule globally, it is highly effective at reducing the fleet size, which directly minimizes the total cost. In contrast, the DQN agent, due to its sequential constructive nature,

| Dataset   | Total Cost |          | Distance |         | Vehicles |      | Energy (V2G) |        | Time (s) |      |
|-----------|------------|----------|----------|---------|----------|------|--------------|--------|----------|------|
|           | PSO        | RDQN     | PSO      | RDQN    | PSO      | RDQN | PSO          | RDQN   | PSO      | RDQN |
| c101_100  | 7650.28    | 7774.59  | 1350.28  | 774.59  | 9        | 10   | 0.00         | 0.00   | 2.11     | 5.93 |
| c101_25   | 2540.68    | 2958.42  | 440.68   | 237.12  | 3        | 5    | 0.00         | 181.09 | 0.63     | 0.42 |
| c101_50   | 3506.87    | 3826.36  | 706.87   | 326.36  | 4        | 5    | 0.00         | 0.00   | 0.80     | 1.61 |
| c102_100  | 7650.28    | 8751.67  | 1350.28  | 1051.67 | 9        | 11   | 0.00         | 0.00   | 1.42     | 6.08 |
| c102_25   | 2388.70    | 2724.30  | 288.70   | 241.47  | 3        | 4    | 0.00         | 73.76  | 0.62     | 0.43 |
| c102_50   | 3337.30    | 5328.47  | 537.30   | 428.47  | 4        | 7    | 0.00         | 0.00   | 0.79     | 1.64 |
| r101_100  | 6186.87    | 18819.38 | 1286.87  | 1537.81 | 7        | 25   | 0.00         | 50.80  | 1.98     | 7.33 |
| r101_25   | 1775.47    | 6030.04  | 375.47   | 556.78  | 2        | 8    | 0.00         | 29.47  | 0.50     | 0.92 |
| r101_50   | 3576.94    | 10647.94 | 776.94   | 1018.25 | 4        | 14   | 0.00         | 39.61  | 0.79     | 1.77 |
| r102_100  | 6171.54    | 18513.12 | 1271.54  | 1529.11 | 7        | 25   | 0.00         | 120.00 | 1.26     | 5.75 |
| r102_25   | 1775.47    | 6095.95  | 375.47   | 585.58  | 2        | 8    | 0.00         | 20.85  | 0.50     | 0.42 |
| r102_50   | 3593.61    | 10262.09 | 793.61   | 958.14  | 4        | 14   | 0.00         | 115.36 | 0.78     | 1.58 |
| rc101_100 | 7126.69    | 15286.38 | 1526.69  | 1666.58 | 8        | 20   | 0.00         | 88.42  | 2.06     | 5.77 |
| rc101_25  | 2528.73    | 3616.85  | 428.73   | 341.74  | 3        | 5    | 0.00         | 52.30  | 0.59     | 0.43 |
| rc101_50  | 4326.94    | 7634.86  | 826.94   | 755.30  | 5        | 10   | 0.00         | 28.01  | 0.85     | 1.66 |
| rc102_100 | 7108.06    | 13467.22 | 1508.06  | 1572.96 | 8        | 17   | 0.00         | 1.34   | 1.37     | 8.09 |
| rc102_25  | 2650.89    | 3735.34  | 550.89   | 355.78  | 3        | 5    | 0.00         | 28.01  | 0.60     | 0.42 |
| rc102_50  | 4278.02    | 6268.44  | 778.02   | 668.44  | 5        | 8    | 0.00         | 0.00   | 0.84     | 1.61 |

**Table 2**  
Comparison of PSO and Rainbow DQN Algorithms on Solomon Instances

often terminates routes conservatively to avoid battery depletion, ultimately increasing the number of vehicles deployed.

## 7.2 Fleet Utilization

A substantial difference is observed in the number of vehicles used by the two methods. PSO achieves an average fleet size of 5.0 vehicles, whereas Rainbow DQN deploys an average of 11.17 vehicles. The DQN policy exhibits a cautious route completion strategy, wherein the agent often returns to the depot earlier than necessary to avoid infeasible states. While this leads to safer routing, it significantly increases the number of vehicles, which subsequently escalates the overall cost. PSO, by contrast, compresses customer assignments more efficiently across fewer vehicles.

## 7.3 Distance Efficiency

Despite its larger fleet size, the Rainbow DQN demonstrates superior efficiency in minimizing travel distance. On average, the total distance covered in the DQN-generated solutions is 3.92% lower than that in PSO solutions. This suggests that the reinforcement learning agent has successfully learned local navigation patterns and short-path behaviors. However, this improvement in distance does not compensate for the much larger fleet size, which dominates the total cost.

## 7.4 V2G Energy Utilization

An important distinction between the two methods emerges in their handling of Vehicle-to-Grid (V2G) discharge opportunities. Rainbow DQN actively utilizes V2G discharge stations and generates a total of 829.02 kWh of discharged energy. In contrast, the PSO algorithm does not discharge energy at all across the eval-

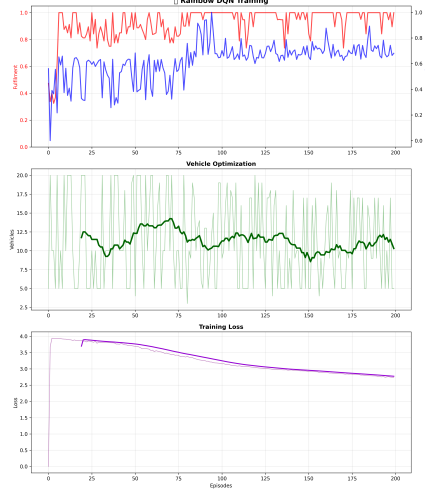
uated instances. This indicates that the reinforcement learning agent is more responsive to immediate reward feedback from energy revenue. PSO likely avoids discharge stations because the time penalties associated with visiting them outweigh the revenue gains within the global optimization landscape.

## 7.5 Computational Performance

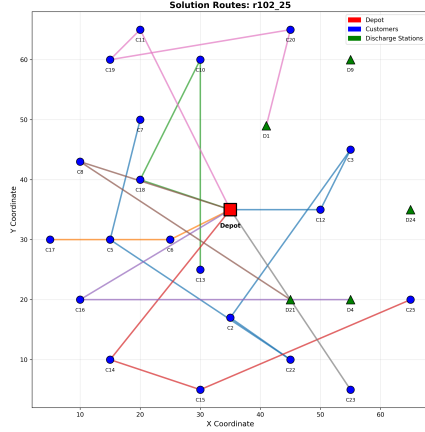
In terms of computational time, PSO exhibits faster runtime performance with an average execution time of 1.03 seconds, while DQN requires 2.88 seconds per instance. Although DQN is slower, its inference time remains near real-time and suitable for practical applications. The additional time arises from step-by-step neural network evaluations, whereas PSO evaluates complete solution candidates in parallel.

## 7.6 Summary

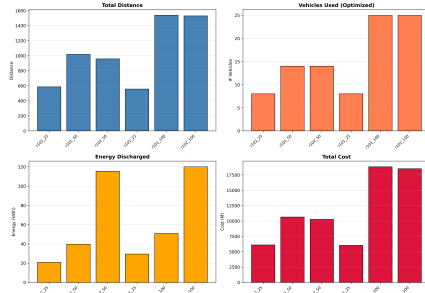
Overall, PSO emerges as the more cost-efficient and operationally scalable method due to its superior fleet optimization capabilities. Rainbow DQN, however, excels in capturing fine-grained routing behaviors such as minimizing route distance and exploiting V2G revenue opportunities. These findings suggest that PSO is better suited for strategic planning scenarios where minimizing vehicle count is critical, while Rainbow DQN holds promise for dynamic or real-time operational environments, especially those involving variable energy pricing and real-time decision-making. A hybridization of both approaches may potentially combine their strengths and yield further improvements.



(a) Rainbow DQN Learning Curve



(b) Solution Route r102\_25



(c) Results Comparison

Figure 1. RL Training and Testing

## 8 Conclusion

This work successfully established a multi-service framework for last-mile logistics, demonstrating the operational feasibility of integrating Vehicle-to-Grid (V2G) services into standard Electric Vehicle (EV) delivery routes. By formulating the Constrained Electric Vehicle Routing Problem with Time Windows and Discharge (CEVRPTW-D), we modeled a system in which logistics fleets not only fulfill delivery demands but also generate revenue by supporting the electrical grid during peak pricing intervals 5; 23.

Our investigation employed a three-tier methodological approach to address the complexities of battery management, time windows, and grid synchronization:

1. **Exact Optimization (MILP):** A Mixed-Integer Linear Programming model was developed to define the theoretical optimum. However, computational experiments confirmed that this exact approach is constrained by scalability limitations, successfully solving only small-scale instances of up to 10 nodes before encountering exponential computational growth due to the NP-hard structure of the problem 6; 25.
2. **Strategic Efficiency (PSO):** The Particle Swarm Optimization metaheuristic proved highly effective in minimizing operational costs. It achieved near-optimal solutions with the smallest fleet size, making it the most suitable method for static, cost-driven daily route planning 8; 125.
3. **Dynamic Intelligence (Rainbow DQN):** The Deep Reinforcement Learning framework (Rainbow DQN) demonstrated strong capability in managing energy constraints and leveraging V2G opportunities. Unlike the heuristic approach, the DQN agent successfully identified profitable discharge intervals, validating its potential for real-time decision-making in smart-grid environments 8; 272.

**Key Findings:** The results reveal a fundamental trade-off in sustainable logistics design. While metaheuristic optimization methods such as PSO excel in reducing capital expenditure through fleet minimization, learning-based models such as Rainbow DQN exhibit superior flexibility in handling complex energy dynamics and enhancing V2G revenue generation. Together, these approaches highlight the complementary strengths of strategic planning and dynamic decision-making in modern EV-based logistics.

**Future Directions:** Future work will focus on bridging the gap between deterministic modeling and real-world stochasticity. This includes integrating uncertain traffic patterns, dynamic electricity pricing, multi-depot operations, and heterogeneous EV fleets to enhance the robustness, scalability, and economic viability of grid-interactive logistics systems 36; 336. Expanding the reinforcement learning framework to incorporate multi-agent coordination and predictive energy pricing models also presents a promising direction for further research.

The findings validate the potential of V2Genabled routing to generate additional economic value without jeopardizing logistics feasibility. Reinforcement learning emerges as a promising direction for intelligent fleet management in dynamic smart-grid-aware logistics environments.

Future work includes extending the model to support stochastic travel times and energy consumption, dynamic electricity pricing, heterogeneous vehicle fleets, multiple depots, and real traffic network integration to enhance deployment realism and robustness.

## References

- Bruni, M. E., O. Jabali, and S. Khodaparasti (Mar. 2023). "The Electric Vehicle Route Planning Problem with Energy Consumption Uncertainty". In: *Transportation Research Part B: Methodological* 172, pp. 79–102. doi: 10.1016/j.trb.2023.02.004.
- Narayanan, Ajay et al. (Sept. 2024a). "A Reinforcement Learning Approach for Electric Vehicle Routing Problem with Vehicle-to-Grid Supply". In: *Proceedings of the International Conference on OR for Mobility (OR4Mobility)* 1, pp. 1–10. doi: 10.48550/arXiv.2409.XXXXX. arXiv: 2409.XXXXX [cs.AI].

Narayanan, Ajay et al. (Sept. 2024b). “Agent-based Learning Approach to Electric Vehicle Routing Problem with Vehicle-to-Grid Supply and Battery Swapping”. In: *Proceedings of the International Conference on OR for Mobility (OR4Mobility)* 1, pp. 11–20. doi: 10.48550/arXiv.2409.XXXXX. arXiv: 2409.XXXXX [cs.AI].